

Electricity price spike forecasting via multivariate adaptive visibility graph neural networks

Tianshu Bao ^{a,b,*}, Nariman Mahdavi ^b, Chris Mccarthy ^a, Dana Rezazadegan ^a

^a Swinburne University of Technology, Hawthorn, Melbourne, 3122, VIC, Australia

^b CSIRO Energy, 10 Murray Dwyer Circuit, Mayfield West, 2304, Newcastle, NSW, Australia

ARTICLE INFO

Keywords:

Electricity price spikes forecasting
Adaptive visibility graph (AVG)
Multivariate time series
Graph neural network (GNN)
Oversampling

ABSTRACT

Accurate forecasting of electricity price spikes is critical for energy market risk management, as these rare yet disruptive events cause substantial financial losses and arise from complex interactions among multiple market variables. An enhanced Graph Neural Network (GNN) framework is proposed that captures interdependencies among these Multivariate variables via an Adaptive Visibility Graph (MAVG) network. A hierarchical Differentiable Pooling (DiffPool) GNN architecture is then applied to coarsen this graph and forecast the next-step price spike probability. To address the extreme class imbalance inherent in spike data, a novel oversampling method called the balanced epoch sampler is also introduced. The proposed MAVG-DiffPool framework with oversampling is validated using real electricity prices from Australia under multiple spike thresholds. The framework achieves an F1-score of 0.84 under the moderate threshold. Under the extreme threshold, the framework outperformed LSTM by achieving nearly double the F1-score, demonstrating its effectiveness and robustness in addressing the challenge of spike forecasting.

1. Introduction

In recent decades, volatility in electricity prices has increased in countries with deregulated electricity markets [1]. A significant and disruptive aspect of electricity price volatility is price spikes. These are sudden, short-duration events characterised by a price reaching hundreds or even thousands of times above normal levels [2]. Price spikes result from a range of factors, including unexpected power outages, supply-demand imbalances, and extreme weather events [3]. For example, in June 2022 [4], the Australian National Electricity Market (NEM) experienced sustained high prices, resulting in wholesale costs exceeding A\$1.3 million over a seven-day period. This triggered an administrative price cap of A\$300/MWh, leading several generators to withdraw capacity. The supply shortage prompted the Australian Energy Market Operator (AEMO) to suspend the market. This incident highlights the significant financial risks for market participants and threats to system reliability [3,5].

Existing literature has made progress in electricity price spike forecasting using both traditional and deep learning approaches. Early approaches to spike forecasting relied on traditional machine learning methods [6,7]. These models required explicit modelling structures and manual feature selection, limiting their flexibility in capturing nonlinear

or long-term temporal dependencies. Deep learning models have demonstrated significant advantages in electricity price spike forecasting [8,9]. Recent studies show Long Short-Term Memory (LSTM) models outperforming traditional tree-based methods [10] in spike forecasting. However, these sequential models learn multivariate interdependencies implicitly through hidden states, lacking explicit structural encoding of such relationships. For spike forecasting, modelling these interdependencies is crucial, as spikes are triggered by complex multivariate interactions involving both contemporaneous and lagged effects [11]. Graph-based representations offer a complementary paradigm by directly encoding interactions as network topology [12]. This motivates the use of Graph Neural Networks (GNNs) for learning from such graph-structured data.

Recent advances in temporal GNNs have effectively modelled time-evolving relational data across various domains. Approaches such as Spatial-Temporal Attention-based GNN [13], Spatial-Temporal Anomaly Transformer [14], and Attentional Spatial-Temporal Aggregation Framework [15] have combined graph convolution with temporal mechanisms. These methods employ techniques including recurrent units and multi-level attention to capture both structural and temporal dependencies in dynamic networks. However, these approaches rely on domain-specific rules and prior knowledge to define node connectivity

* Corresponding author.

E-mail addresses: cbao@swin.edu.au (T. Bao), nariman.mahdavi@csiro.au (N. Mahdavi), cdmccarthy@swin.edu.au (C. Mccarthy), drezazadegan@swin.edu.au (D. Rezazadegan).

<https://doi.org/10.1016/j.epsr.2025.112695>

Received 31 August 2025; Received in revised form 15 November 2025; Accepted 26 December 2025

Available online 16 January 2026

0378-7796/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

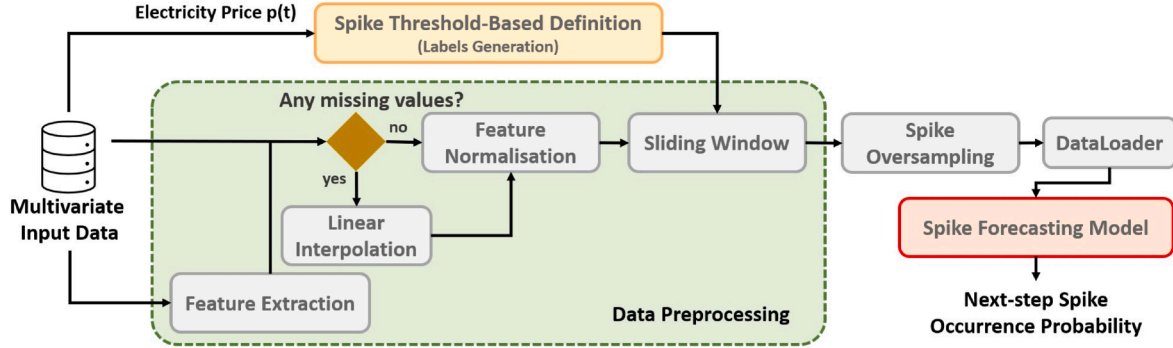


Fig. 1. The overview of the proposed spike forecasting framework.

(e.g., predefined transaction relationships or known graph topologies). In electricity price forecasting, existing GNN-based approaches mainly focus on spatial topological structures [16]. In these models, nodes represent different markets and edges capture spatial relationships such as transmission connections or regional price correlations [17,18]. However, these methods overlook the interdependencies among variables within a regional market. They do not, therefore, capture the complex interaction among multiple variables (such as demand, generation capacity, etc.) that collectively drive price dynamics. This paper addresses this gap by explicitly modelling variable interdependencies for electricity price spike forecasting.

Visibility graph (VG) methods [19] directly construct a graph topology from time series data and eliminating the need for domain-specific rules or prior knowledge. In such graphs, nodes represent temporal observations, while edges encode visibility relationships between time points based on different visibility criteria [20]. These methods initially focused on univariate approaches that capture temporal dependencies, and have since been extended to multivariate settings. Multiplex Visibility Graphs (MVGs) [21] connect variables only at contemporaneous timestamps, while multilayer horizontal visibility graphs [22] extend this to capture lagged cross-correlations through cross-horizontal visibility. However, both models employ fixed visibility criteria (e.g. horizontal visibility) for graph construction that cannot be adapted or optimised for specific forecasting applications.

To enable task-specific adaptation, the Adaptive Visibility Graph (AVG) [23] replaces fixed visibility criteria with learnable neural network layers. However, it remains limited to univariate settings. This paper addresses this limitation by proposing a multivariate extension. The proposed framework is evaluated on the NEM Victoria region, characterised by substantial wind and solar generation and high price volatility, making spike forecasting particularly challenging. Our main contributions are as follows.

- A Multivariate Adaptive Visibility Graph (MAVG) is proposed, inspired by the AVG framework. It adaptively captures multivariate interdependencies and enables end-to-end learning of graph topology for GNN-based forecasting.
- A deterministic oversampling algorithm is proposed to address the severe class imbalance in spike forecasting, where spike occurrences are significantly less common than normal. This method eliminates the stochastic variance of traditional random sampling, ensuring stable training convergence and complete data coverage.
- A novel electricity price spike forecasting framework, MAVG-DiffPool, is proposed that integrates data embedding, MAVG, and Differentiable Pooling (DiffPool) architecture. This integration enables comprehensive node representations, provides effective graph-based modelling of temporal and multivariate interdependencies for spike forecasting.

The rest of the paper is organised as follows. Section 2 introduces the forecasting framework, including the dataset, spike definitions, pre-processing, oversampling algorithm and the proposed MAVG-DiffPool model. Section 3 presents the implementation details, while Section 4 provides the results and discussions. It includes the ablation studies to assess individual contributions of the key innovations, and a comparative analysis against established baseline methods for spike forecasting. Finally, Section 5 summarises the key findings and outlines future research directions.

2. Spike forecasting framework

This section outlines the price spike forecasting framework, see Fig. 1. The framework processes multivariate input data through two parallel pathways: First, given the price time series $p(t)$, a binary spike label (y_{label}) is generated using a spike threshold-based definition. Second, the data preprocessing extracts input features ($\mathbf{X}_{\text{multi}}, \mathbf{X}_{\text{cal}}$) and target variable (y_{label}). Subsequently, a balanced sampler index I_{bal} is generated via oversampling, and the dataloader then retrieves batches for model training.

2.1. Multivariate input data

The multivariate input data covers a three-year period from 1 January 2017 to 31 December 2019. The data is sourced from the NEM Victoria region at 30-min trading intervals. The dataset comprises market data (electricity price, scheduled demand, scheduled generation, semi-scheduled generation from intermittent renewable sources) from AEMO,¹ and weather data (temperature and wind speed) from the Australian Bureau of Meteorology.² Summary statistics for these external variables are provided in Table 1, while electricity price characteristics are analysed in Section 3.1.

2.2. Price spike threshold-based definition

Price spikes are defined using a threshold-based binary indicator function,

$$y_{\text{label}}(t) = \begin{cases} 1 & \text{if } p(t) > \tau(t), \\ 0 & \text{otherwise,} \end{cases} \quad t \in \{1, \dots, T\}, \quad (1)$$

where $p(t)$ is the price at time t and $\tau(t)$ is the threshold function. Both fixed and variable thresholds are employed. For fixed thresholds, the A\$100/MWh threshold enables peaking generators to compete with baseload generators [24], while the A\$300/MWh threshold represents

¹ Australian Energy Market Operator: <https://aemo.com.au/energy-systems/electricity/national-electricity-market-nem/data-nem/market-data-nemweb>

² Australian Bureau of Meteorology: <http://www.bom.gov.au/>

Table 1
Summary statistics of external variables.

Variable	Mean	SD	Min	Max	Missing	Source
Temperature (°C)	13.76	6.66	-2.55	42.55	1	BoM
Wind Speed (km/h)	19.25	9.04	0.00	69.30	2	BoM
Scheduled Demand (MWh)	4,964.05	869.01	3,058.00	9,310.69	0	AEMO
Scheduled Generation (MWh)	4,944.95	806.23	2,875.00	9,395.02	0	AEMO
Semi-Scheduled Generation (MWh)	363.87	283.32	0.00	1,497.64	0	AEMO
Supply-Demand Ratio	-0.11	0.15	-0.77	0.27	0	Calculated

Note: SD, Standard Deviation; BoM, Bureau of Meteorology. Temperature and wind speed are regional averages (Melbourne and Ballarat).

Table 2
Calendar features extracted from datetime index.

Feature	Range	Description
Minutes	{0, 30}	Minutes component (30-min intervals)
Hour	1–24	Hour of day
Day of Week	1–7	Day of week (Monday = 1, Sunday = 7)
Day of Month	1–31	Calendar day of month
Day of Year	1–366	Julian day (accounts for leap years)
Month	1–12	Calendar month
Weekend Indicator	{0, 1}	Binary flag (1 for Saturday/Sunday)

Note: All features are extracted directly from the pandas datetime index.

the severe spike boundary [25]. For variable thresholds, $\tau(t)$ is defined as $\mu_t + k\sigma_t$, where μ_t and σ_t are computed using a 7-day rolling window, and $k = 3$ following [9].

2.3. Data preprocessing

Feature extraction and linear interpolation. To capture market dynamics beyond electricity prices, a supply-demand index (SDI) [26] is constructed at time t ,

$$\text{SDI}(t) = (D_t - G_t)/D_t, \quad (G_t = G_t^{\text{sch}} + G_t^{\text{semi}}), \quad (2)$$

where D_t denotes scheduled demand, G_t^{sch} and G_t^{semi} represent scheduled and semi-scheduled generation, respectively. A positive $\text{SDI}(t)$ indicates a supply shortfall (excess demand). Such conditions are typically associated with price volatility and potential spike occurrence. The multivariate input features $\mathbf{x}_{\text{multi}}^j, j \in \{1, \dots, M\}$ comprise market and weather variables, along with the derived SDI. The calendar features $\mathbf{x}_{\text{cal}}^k, k \in \{1, \dots, N\}$ capture temporal patterns at multiple scales, as detailed in Table 2. Missing values in both calendar and multivariate features are handled through linear interpolation.

Feature normalisation. Multivariate input features are normalised using standard scaling (z-score normalisation),

$$\mathbf{x}_{\text{multi}}^j = (\mathbf{x}_{\text{multi}}^j - \mu^j)/\sigma^j, \quad j \in \{1, \dots, M\}, \quad (3)$$

where μ^j and σ^j are the mean and standard deviation of the j th feature. Calendar features are scaled to the range $[-0.5, 0.5]$ by min-max normalisation,

$$\mathbf{x}_{\text{cal}}^k = \frac{\mathbf{x}_{\text{cal}}^k - \min(\mathbf{x}_{\text{cal}}^k)}{\max(\mathbf{x}_{\text{cal}}^k) - \min(\mathbf{x}_{\text{cal}}^k)} - 0.5, \quad k \in \{1, \dots, N\}, \quad (4)$$

Sliding window. The sliding window technique transforms features into structured input-output pairs. A window of length S (look-back period) slides through T time steps. The window size $S = 12$ is determined through autocorrelation function (ACF) analysis of 2017–2018 price data, which identifies the smallest significant lag at 12 steps [27]. The robustness of this choice is validated through sensitivity analyses across alternative window sizes. The sliding window process can be expressed

as,

$$\begin{cases} \mathbf{X}_{\text{multi}}[t, i, j] = \mathbf{x}_{\text{multi}}^j[t - S + i], \\ \mathbf{X}_{\text{cal}}[t, i, k] = \mathbf{x}_{\text{cal}}^k[t - S + i], \end{cases} \quad (5)$$

$$t \in \{S, S+1, \dots, T-1\},$$

$$i \in \{1, \dots, S\}.$$

For each time step t , the window captures the previous S time steps, with i indexing the position within the window. $\mathbf{X}_{\text{multi}} \in \mathbb{R}^{(T-S) \times S \times M}$ contains the processed multivariate features, while $\mathbf{X}_{\text{cal}} \in \mathbb{R}^{(T-S) \times S \times N}$ contains the calendar features. $\mathbf{y}_{\text{label}}(t)$ is the binary indicator for price spike occurrence at the next step $t+1$, predicted using the window $[t-S+1, \dots, t]$. The target labels are organised as $\mathbf{y}_{\text{label}} \in \mathbb{R}^{(T-S) \times 1}$.

2.4. Spike oversampling: balanced epoch sampler

Price spikes cause a severe class imbalance, which biases machine learning toward the dominant normal classes [28]. To address this, a deterministic balanced epoch sampler (BES) is introduced that generates oversampled index sequences $\mathcal{I}_{\text{bal}}$ from $\mathbf{y}_{\text{label}} \in \mathbb{R}^{(T-S) \times 1}$. Unlike PyTorch's Weighted Random Sampler (WRS) [29], which applies probabilistic resampling without guaranteeing per-epoch balance, the proposed BES method, see Algorithm 22, enforces exact class proportions and ensures complete coverage of the normal class instances within each epoch. The method partitions the label indices into class-specific index sets $\mathcal{I}_c = \{i : \mathbf{y}_{\text{label}}[i] = c\}$ for $c \in \{0, 1\}$, where $c = 0$ denotes the normal (majority) class and $c = 1$ denotes the spike (minority) class. It then constructs balanced batches of size B by sampling $s = \text{floor}(B/2)$ instances per class. The normal class is sampled without replacement (each sample can only be selected once) to ensure full coverage, while the spike class is oversampled with replacement. Note that oversampling is based on target labels $\mathbf{y}_{\text{label}}$ rather than window content. As a result, even though sliding window processing creates overlapping windows, each window has different target labels, avoiding heavy duplication.

This design mitigates gradient instability caused by WRS's stochastic batch composition under severe imbalance. Specifically, the selection of spike class samples per batch follows $n_1 \sim \text{Binomial}(B, p)$ where $p = 1/(1 + \text{IR})$. $\text{IR} = n_0/n_1$ denotes the imbalance ratio (ratio of normal to spike class sizes). This yields,

$$\text{Var}\left[\frac{n_1}{B}\right] = \frac{\text{IR}}{B(1 + \text{IR})^2}, \quad (6)$$

This composition variance induces gradient instability and directional noise, leading to the non-monotonic spike class loss dynamics [30]. Under extreme imbalance, the probability of a zero spike batch approaches unity ($\lim_{\text{IR} \rightarrow \infty} P(n_1 = 0) \rightarrow 1$), causing gradient updates to alternate between normal-biased and mixed-class regimes.

The proposed deterministic sampler eliminates composition variance by fixing $n_1 = \lfloor B/2 \rfloor$ for all batches,

$$\text{Var}\left[\frac{n_1}{B}\right] = 0, \quad (7)$$

This ensures consistent per-class gradient contributions across iterations, addressing the composition-induced instability identified in Franczi et al. [30] and yielding more stable convergence dynamics for spike class optimisation.

Algorithm 1: Balanced epoch sampler (BES).

Input: spike labels $\mathbf{y}_{\text{label}} \in \{0, 1\}^{T-S}$ (0: normal, 1: spike), where S is the sliding window size and T is the total time steps, batch size B .

Output: oversampled balanced index sequence $\mathcal{I}_{\text{bal}}$ for each epoch training.

```

// Stage1: Class partitioning and analysis
1 for  $c \in \{0, 1\}$  do
2    $\mathcal{I}_c \leftarrow \{i : \mathbf{y}_{\text{label}}[i] = c\}$ ,  $n_c \leftarrow |\mathcal{I}_c|$ ;
3 end
4  $n_{\text{max}} \leftarrow \max(n_0, n_1)$ ,  $c_{\text{max}} \leftarrow \arg \max_c(n_c)$ ;
// Stage2: Balanced batch generation
// Initialise features
5  $s \leftarrow \text{floor}(B/2)$  // round down, samples per class per batch
6  $K \leftarrow \text{ceil}(n_{\text{max}}/s)$  // round up, required batches
7  $\mathcal{I}_R \leftarrow \mathcal{I}_{c_{\text{max}}}$  // remaining normal sample indices
8  $\mathcal{I}_{\text{bal}} \leftarrow \emptyset$ ;
// Balanced sampling
9 for  $k = 1$  to  $K$  do
10   $\mathcal{I}_B \leftarrow \emptyset$  // initial current batch indices set
11  for  $c \in \{0, 1\}$  do
12    if  $c = c_{\text{max}}$  and  $|\mathcal{I}_R| \geq s$  then
13       $\mathcal{T} \leftarrow$  sample  $s$  indices from  $\mathcal{I}_R$  without replacement // ensure complete coverage
14       $\mathcal{I}_R \leftarrow \mathcal{I}_R \setminus \mathcal{T}$  // remove used samples
15    else
16       $\mathcal{T} \leftarrow$  sample  $s$  indices from  $\mathcal{I}_c$  with replacement // oversample spike class
17    end
18     $\mathcal{I}_B \leftarrow \mathcal{I}_B \cup \mathcal{T}$ ;
19  end
20   $\mathcal{I}_{\text{bal}} \leftarrow \mathcal{I}_{\text{bal}} \cup \text{shuffle}(\mathcal{I}_B)$ ;
21 end
22 return  $\mathcal{I}_{\text{bal}}$ ;

```

2.5. DataLoader processing

After constructing the oversampled balanced index set $\mathcal{I}_{\text{bal}}$, PyTorch's DataLoader is applied to generate balanced batches. The DataLoader uses $\mathcal{I}_{\text{bal}}$ as a custom sampler to extract samples from the sliding window outputs $(\mathbf{X}_{\text{multi}}, \mathbf{X}_{\text{cal}}, \mathbf{y}_{\text{label}})$. For each index $i \in \mathcal{I}_{\text{bal}}$, the DataLoader retrieves the samples $\mathbf{X}_{\text{multi}}[i, :, :]$, $\mathbf{X}_{\text{cal}}[i, :, :]$, and $\mathbf{y}_{\text{label}}[i, :]$. These samples are then grouped into batches of size B . This process produces balanced training batches $\tilde{\mathbf{X}}_{\text{multi}} \in \mathbb{R}^{B \times S \times M}$, $\tilde{\mathbf{X}}_{\text{cal}} \in \mathbb{R}^{B \times S \times N}$, and $\tilde{\mathbf{y}}_{\text{label}} \in \mathbb{R}^{B \times 1}$ for each epoch.

To ensure a valid evaluation, oversampling is not applied to the validation and testing datasets, maintaining their original class distributions. The balanced batches serve as inputs $(\tilde{\mathbf{X}}_{\text{multi}}, \tilde{\mathbf{X}}_{\text{cal}})$ and targets $(\tilde{\mathbf{y}}_{\text{label}})$ for model training, as illustrated in Fig. 1.

2.6. Spike forecasting model: MAVG-DiffPool

Fig. 2 illustrates the spike forecasting model in three stages. First, all input features from each time step are put into a shared embedding space to generate node features. Next, MAVG constructs an adaptive adjacency matrix from the multivariate features by computing the visibility score for each edge between any two time steps. Each edge encodes both the temporal relationship and the multivariate interdependencies across features. Finally, DiffPool hierarchically coarsens the resulting graph to predict the next-step spike probability.

Data embedding. It integrates three inputs that provide essential information for modelling price spike behaviour. The first two are the previously introduced oversampled multivariate features $\tilde{\mathbf{X}}_{\text{multi}} \in \mathbb{R}^{B \times S \times M}$ and oversampled calendar features $\tilde{\mathbf{X}}_{\text{cal}} \in \mathbb{R}^{B \times S \times N}$. Additionally, positional indices $\tilde{\mathbf{X}}_{\text{pos}} \in \mathbb{R}^{B \times S \times 1}$ with values ranging from 1 to S represent the sequential position of each time step within the sliding window. Here, B, S, M, N denote batch size, sequence length, number of multivariate features, and number of calendar features, respectively. Following the uniform input representation from Informer [31], these inputs are embedded as initial node representations $\mathbf{H}^0 \in \mathbb{R}^{B \times S \times d_{\text{model}}}$ in the graph,

$$\mathbf{H}^0 = \text{Conv}(\tilde{\mathbf{X}}_{\text{multi}}) + \sum_{j=1}^N \text{Emb}(\tilde{\mathbf{X}}_{\text{cal}}^j) + \text{PE}(\tilde{\mathbf{X}}_{\text{pos}}), \quad (8)$$

where $\text{Conv}(\cdot)$ denotes the convolutional layers that project inputs to model dimension d_{model} . $\text{Emb}(\cdot)$ represents learnable embedding layers that map discrete calendar features to continuous vector representations. $\text{PE}(\cdot)$ represents sinusoidal position encoding [32] that transforms each position $\text{pos} \in \tilde{\mathbf{X}}_{\text{pos}}$ to d_{model} ,

$$\text{PE}(\text{pos}, i) = \begin{cases} \sin(\text{pos}/10000^{2i/d_{\text{model}}}), & \text{if } i \text{ is even,} \\ \cos(\text{pos}/10000^{2i/d_{\text{model}}}), & \text{if } i \text{ is odd,} \end{cases} \quad (9)$$

$$i \in \{0, 1, \dots, d_{\text{model}}/2 - 1\}.$$

MAVG construction. AVG has demonstrated that learnable convolutional operations can effectively generalise the fixed visibility criteria of traditional VGs [23]. Building upon this validated principle, MAVG extends it to multivariate time series. Unlike multivariate VGs (e.g., MVGs [33], Fig. 3a) that rely on fixed visibility criteria, MAVG (Fig. 3b) replaces these criteria using one-dimensional (1D) convolutional layers.³ These layers operate across M channels, where each channel corresponds to a distinct feature (e.g., price, demand). For each kernel size $s \in \{2, \dots, S\}$, a convolutional layer computes adaptive visibility scores $\mathbf{V}_t^s$ by aggregating cross-channel dependencies, which define edges between time points t and $t + s - 1$, i.e.,

$$\mathbf{V}_t^s[b] = \text{ReLU}(\mathbf{c}_t^s[b]), \quad t \in \{1, \dots, S + 1 - s\}, b \in \{1, \dots, B\} \quad (10a)$$

$$\mathbf{c}_t^s[b] = \text{Conv1d}_M^{(s)}(\tilde{\mathbf{X}}_{\text{multi}}^T[b]) = \sum_{j=1}^M \sum_{i=1}^s w_{j,i}^s \cdot \mathbf{x}_{j,t+i-1}[b], \quad (10b)$$

where $w_{j,i}^s$ are learnable kernel weights, and the nonlinear activation function ReLU [35] is applied to filter out negative values, ensuring non-negative visibility scores. $\tilde{\mathbf{X}}_{\text{multi}}^T \in \mathbb{R}^{B \times M \times S}$ is the transpose of $\tilde{\mathbf{X}}_{\text{multi}} \in \mathbb{R}^{B \times S \times M}$ and $\mathbf{x}_{j,t+i-1}[b]$ denotes the observation at time step $t + i - 1$ of the j th channel. Then $\mathbf{V}_t^s$ populates a symmetric adjacency matrix $\mathbf{A} \in \mathbb{R}^{B \times S \times S}$,

$$\mathbf{A}[b, t + s - 1, t] = \mathbf{A}[b, t, t + s - 1] = \mathbf{V}_t^s[b], \quad \mathbf{A}[b, t, t] = 0. \quad (11)$$

Specifically, when $M = 1$, Eq. (10b) reduces to the univariate case (AVG), where the convolutional aggregation $\sum_{i=1}^s$ processes a single time series through different kernel lengths s . For $M > 1$, each $\mathbf{V}_t^s$ aggregates dependencies across all M channels through the summation $\sum_{j=1}^M$ (e.g. how price fluctuates from t to $t + s - 1$ driven by demand and temperature interactions). Therefore, each $\mathbf{V}_t^s[b]$ aggregates $M \times s$ observations compared to s observations in the univariate case. The resulting adjacency matrix $\mathbf{A}$ is directly used in subsequent GNN layers without additional normalisation or sparsification.

MAVG ultimately constructs a single $S \times S$ adjacency matrix, compared to the $(M \times S) \times (M \times S)$ adjacency matrix generated by traditional multivariate VGs [22] (e.g., MVGs, shown in Fig. 3a). This adaptive graph construction maintains computational efficiency while cap-

³ In convolutional neural networks [34], channels represent the depth of the input, and kernel size defines the temporal window length for each convolution operation.

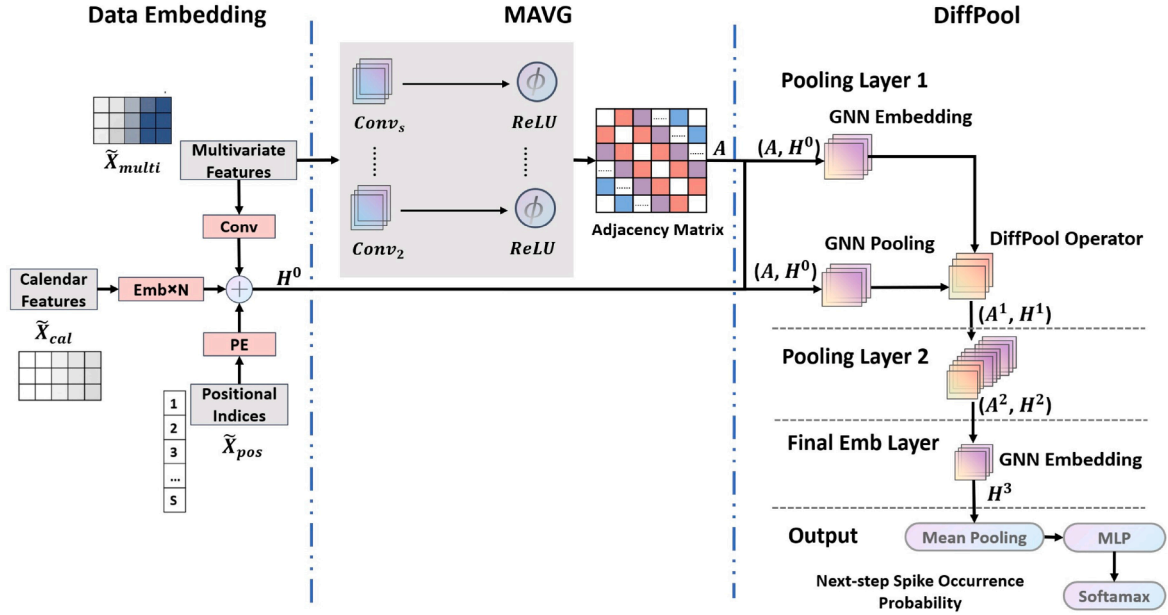
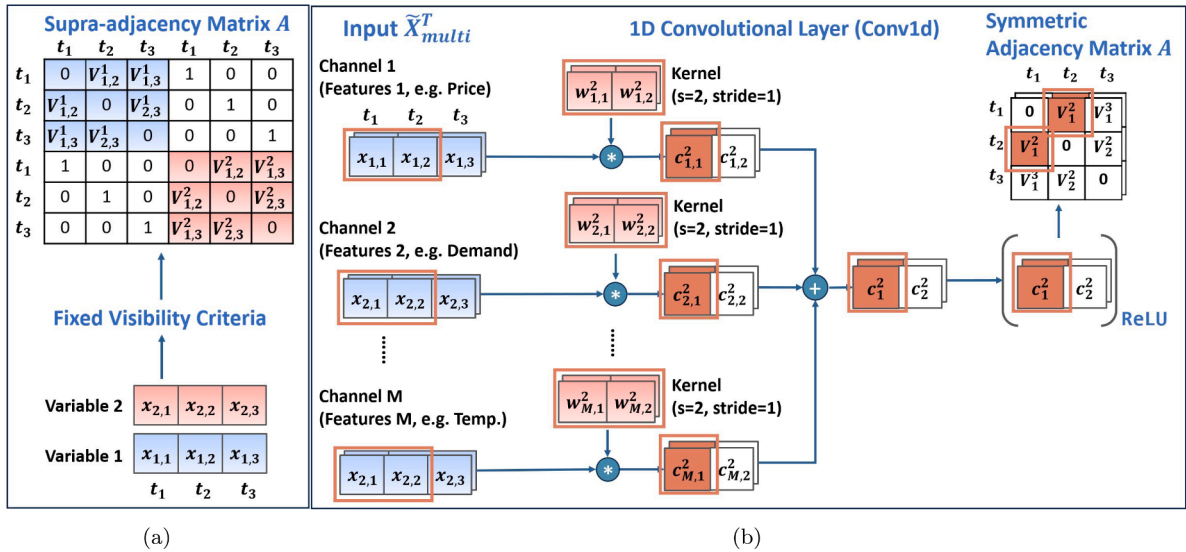


Fig. 2. The overview of the three-stage spike forecasting model (MAVG-DiffPool).

Fig. 3. (a) MVGs graph construction from multivariate time series [22], where $V_{i,j}^a \in \{1, 0\}$. (b) The proposed MAVG with a sequence length of $S = 3$ and a kernel size of $s = 2$ per batch.

turing cross-channel dependencies. It is suitable for price spike forecasting, as these spikes arise from multivariate interdependencies [11].

DiffPool architecture. The initial graph representation (adjacency matrix A and node embeddings H^0) feeds into a hierarchical DiffPool architecture [36] with $L = 2$ pooling layers, followed by a final GNN embedding layer. The architecture pools temporal nodes hierarchically. This captures higher-order dependencies, suppresses fluctuations, and produces robust spike-predictive representations. Each layer $l \in \{1, \dots, L\}$ reduces the number of nodes from N_{l-1} to N_l ($N_0 = S$) with a pooling ratio of 0.25. The two-layer design strikes a balance between computational efficiency and information retention, as a single pooling layer provides insufficient abstraction, while excessive layers risk losing critical temporal details. Each graph pooling layer is achieved through three key components.

The first component, GNN Embedding (see Fig. 2) employs a multi-layer Graph Sample and Aggregate (GraphSAGE) [37] to learn enriched node embeddings. At each pooling layer l , node embeddings are com-

puted independently for each batch as,

$$\mathbf{Z}^l = \text{GraphSAGE}_{\text{embed}}(\mathbf{A}^{l-1}, \mathbf{H}^{l-1}), \quad (12)$$

where $\mathbf{A}^{l-1} \in \mathbb{R}^{B \times N_{l-1} \times N_{l-1}}$ represents the adjacency matrix at pooling layer $l-1$, and $\mathbf{H}^{l-1} \in \mathbb{R}^{B \times N_{l-1} \times d_{\text{model}}}$ is the matrix of node features from the previous pooling layer. For $l = 1$, $\mathbf{A}^0 = \mathbf{A}$ is the adjacency matrix constructed by the MAVG method, and $\mathbf{H}^0$ denotes the initial node features. For $l > 1$, $\mathbf{A}^{l-1}$ is the coarsened adjacency matrix generated by the preceding pooling layer. Each row z_t^l of $\mathbf{Z}^l$ corresponds to node $t \in \{1, \dots, N_{l-1}\}$ and is updated as,

$$z_t^l = \mathbf{W}_{\text{root}}^l \cdot h_t^{l-1} + \mathbf{W}_{\text{rel}}^l \cdot \frac{1}{|\mathcal{N}(t)|} \sum_{u \in \mathcal{N}(t)} h_u^{l-1}, \quad \mathcal{N}(t) = \{u : \mathbf{A}^{l-1}[b, t, u] \neq 0\}, \quad (13)$$

where $\mathbf{W}_{\text{root}}^l$ and $\mathbf{W}_{\text{rel}}^l$ are learnable weight matrices for self-node transformation and neighbour aggregation, respectively. The neighbourhood $\mathcal{N}(t)$ includes all nodes connected to node t in the adjacency matrix

Table 3

Statistical summary of electricity prices (A\$/MWh) by data subset.

Subset	Mean	Min	Max	SD	Skew	Kurt	Count
Training	93.05	−554.62	12,931.04	138.17	62.22	4,873.85	34,935
Validation	91.90	−387.56	2,265.31	58.83	17.33	567.02	8734
Test	119.67	−166.08	14,500.00	568.73	23.00	548.69	8879

Note: SD, Standard Deviation; Kurt, Kurtosis; Skew, Skewness.

Table 4

Number of spikes under different threshold definitions.

Defined method	Threshold	Training	Validation	Test
Fixed Threshold	$\geq \text{A\$300/MWh}$	77 (0.22%)	21 (0.24%)	83 (0.93%)
	$\geq \text{A\$100/MWh}$	11,956 (34.23%)	2965 (33.95%)	3112 (35.04%)
Variable Threshold	$\geq \mu_t + 3\sigma_t$	566 (1.62%)	160 (1.83%)	196 (2.21%)

Note: Percentages in parentheses indicate spike rate relative to subset size; μ_t, σ_t computed using 7-day rolling window.

$\mathbf{A}^{l-1}$. The GraphSAGE_{embed} operator is implemented using the Pytorch DenseSAGEConv.

In parallel, the second component (GNN Pooling) computes a soft cluster assignment matrix $\mathbf{Q}^l$ using a dedicated GraphSAGE_{pool} (also implemented by DenseSAGEConv):

$$\mathbf{Q}^l = \text{GraphSAGE}_{\text{pool}}(\mathbf{A}^{l-1}, \mathbf{H}^{l-1}). \quad (14)$$

The third component (DiffPool Operator) performs differentiable graph coarsening using the learned assignments, implemented by Pytorch DenseDiffPool.

$$\begin{cases} \mathbf{H}^l &= (\mathbf{Q}^l)^T \mathbf{Z}^l, \\ \mathbf{A}^l &= (\mathbf{Q}^l)^T \mathbf{A}^{l-1} \mathbf{Q}^l, \end{cases} \quad (15)$$

where $\mathbf{H}^l$ aggregates the node features $\mathbf{Z}^{l-1}$ into coarsened representations based on the assignment weights $\mathbf{Q}^l$. $\mathbf{A}^l$ is the coarsened matrix obtained by transforming $\mathbf{A}^{l-1}$ via the assignment weights $\mathbf{Q}^l$.

The final GNN embedding layer only applied to generate the final node representation $\mathbf{H}^3$. Subsequently, the model applies mean pooling to aggregate node-level features into a graph-level representation, followed by a multi-layer perceptron (MLP) and softmax activation to forecast the next-step spike probability.

3. Implementation details

3.1. Data partitioning strategy

The multivariate input data were partitioned as follows. Six non-consecutive months from 2019 were randomly selected (fixed seed) as the test set. To prevent data leakage, a 6-h buffer period ($S = 12$ at 30-min intervals) was applied around each test month, excluding affected samples from training and validation. The remaining data from 2017 to 2018 and non-test months of 2019 were chronologically split into training (80%) and validation (20%) sets. The partitioning details are summarised in Table 3.

Table 4 summarises spikes by various definition methods. A fixed threshold of $\geq \text{A\$100/MWh}$ identifies about 35% of the test subset (moderate threshold), while the $\geq \text{A\$300/MWh}$ threshold identifies < 1% (extreme threshold). The variable threshold ($\geq \mu_t + 3\sigma_t$) identifies approximately 2.2% of observations as spikes, adapting to time-varying volatility.

3.2. Model configuration

Bayesian optimisation was conducted using Optuna [38] for 100 evaluations per model. Table 5 presents optimal hyperparameter configurations. The search space included model architecture hyperparameters: model dimension (d_{model}), DiffPool layer hidden size (d_{hidden})

and model layer dropout rate (dropout). Training hyperparameters include the learning rate and batch size (B). The proposed framework was trained using AdamW optimiser [39] with weight decay (1×10^{-2}), ReduceLROnPlateau scheduler [29] (factor = 0.3, patience = 2), and gradient clipping (max norm = 1.0). The proposed MAVG-DiffPool framework with BES oversampling comprises 0.41 M parameters. Training was conducted on a single NVIDIA A100 GPU with PyTorch 2.1.0 (CUDA 12.1), achieving an average training time of 16.46 seconds per epoch with peak GPU memory allocation of 0.05 GB.

3.3. Evaluation metrics

Given the inherent class imbalance in electricity price spike forecasting, comprehensive evaluation metrics for imbalanced binary classification tasks are employed. Precision, recall, F1-score, and Balanced Accuracy [40], Matthews Correlation Coefficient (MCC) [41], and Receiver-operating characteristic (ROC) curves and area under the ROC curve (AUC) analysis were utilised for performance assessment. See Table 6 for their detailed definitions.

4. Results and discussion

4.1. Ablation studies

Multivariate feature contribution. MAVG-DiffPool outperforms the AVGNet across metrics and thresholds (Table 7). Improvements are observed under the extreme threshold ($\text{IR} = 107$), with recall improving from 0.193 to 0.446 and F1-score from 0.259 to 0.345. The ROC curve (Fig. 4a) shows a steep initial rise for the MAVG-DiffPool model. In contrast, the AVGNet curve remains relatively flat overall. Under the variable threshold ($\text{IR} = 52.8$), recall and balanced accuracy improve by 0.13 and 0.07, respectively. The MAVG-DiffPool ROC curve exhibits a consistently sharper ascent (Fig. 4c). However, under the moderate threshold ($\text{IR} = 2.3$), both curves nearly overlap (Fig. 4e). These results highlight that incorporating multivariate features is particularly beneficial under the extreme threshold.

Oversampling contribution. The impact of the BES is isolated by comparing MAVG-DiffPool to the proposed MAVG-DiffPool framework with BES oversampling, see Table 7. Under the extreme threshold, precision improves by 0.16 and F1-score by 0.07, confirming the effectiveness of oversampling for spike-class enhancement. As shown in Fig. 4a, MAVG-DiffPool with BES oversampling achieves a sharper rise at low False Positive Rate (FPR), while the MAVG-DiffPool overtakes at higher FPR. This indicates that oversampling emphasises early spike forecasting and precision.

Table 5
Summary of Bayesian optimisation result for hyperparameter tuning.

Model	Optimal hyperparameters
AVGNet	$d_{\text{model}} = 512$, $B = 128$, $d_{\text{hidden}} = 64$, learning rate = 0.001, dropout = 0.2, $S = 12$, $L = 2$
MAVG-DiffPool	$d_{\text{model}} = 256$, $B = 256$, $d_{\text{hidden}} = 64$, learning rate = 0.001, dropout = 0.2, $S = 12$, $L = 2$
MAVG-DiffPool + BES	$d_{\text{model}} = 512$, $B = 64$, $d_{\text{hidden}} = 64$, learning rate = 0.0005, dropout = 0.3, $S = 12$, $L = 2$

Note: Search ranges: $d_{\text{model}} \in \{128, 256, 512\}$; batch size $B \in \{32, 64, 128, 256\}$; $d_{\text{hidden}} \in \{32, 64, 128\}$; learning rate $\in [1 \times 10^{-3}, 5 \times 10^{-3}]$, sampled uniformly with step size 5×10^{-4} ; dropout $\in [0.1, 0.5]$; BES: Balanced Epoch Sampler oversampling method, '+' indicates the combination of methods.

Table 6
Summary of the evaluation metrics.

Metric	Formula	Description
Precision	$\frac{TP}{TP + FP}$	Proportion of correctly predicted spike cases among all predicted spike cases
Recall (TPR)	$\frac{TP}{TP + FN}$	Proportion of correctly predicted spike cases among all actual spike cases
F1-Score	$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall
FPR	$\frac{FP}{FP + TN}$	Proportion of actual normal incorrectly classified as spike
Balanced Accuracy	$\frac{TPR + TNR}{2}$, $TNR = 1 - FPR$	Average accuracy across both spike and normal price cases
MCC	$\frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$	Matthews Correlation Coefficient
ROC Curve	-	Plot of TPR vs FPR at various threshold settings
AUC	$\int_0^1 TPR d(FPR)$	Area Under the ROC Curve; measures discrimination ability across all thresholds

Note: True Positive(TP), correctly predicted spike occurrence; True Negatives(TN), accurately predicted normal price periods; False Positive(FP), normal prices erroneously predicted as spikes; False Negative(FN), spike occurrence erroneously predicted as normal; TPR is the True Positive Rate; FPR is the False Positive Rate, Metrics are computed with spike as the positive class.

Table 7
Ablation results comparing AVGNet, MAVG-DiffPool, and MAVG-DiffPool + BES across extreme (IR = 107.28), variable (IR = 52.81), and moderate (IR = 2.31) spike thresholds.

Spike definition	Models	Bal. Acc.	Precision	Recall	F1-score	MCC	AUC
Extreme Threshold (IR = 107.28)	AVGNet	0.59	0.21	0.19	0.20	0.19	0.72
	MAVG-DiffPool	0.71	0.19	0.45	0.27	0.28	0.84
	MAVG-DiffPool + BES	0.73	0.35	0.47	0.40	0.40	0.89
Variable Threshold (IR = 52.81)	AVGNet	0.61	0.26	0.24	0.25	0.23	0.81
	MAVG-DiffPool	0.68	0.40	0.37	0.39	0.37	0.88
	MAVG-DiffPool + BES	0.75	0.41	0.52	0.46	0.45	0.89
Moderate Threshold (IR = 2.31)	AVGNet	0.87	0.77	0.88	0.82	0.72	0.94
	MAVG-DiffPool	0.88	0.83	0.84	0.84	0.75	0.95
	MAVG-DiffPool + BES	0.88	0.79	0.89	0.84	0.75	0.97

Note: IR, Imbalanced Ratio; Bal. Acc., Balanced Accuracy. Extreme threshold corresponds to $\geq$ A\$300/MWh price spikes, moderate threshold to $\geq$ A\$100/MWh price spikes. BES: Balanced Epoch Sampler; '+' indicates the combination of methods.

Cumulative effect. Combining multivariate features with oversampling yields benefits that exceed either individual contributions (see Table 7). Under the extreme threshold, MAVG-DiffPool with BES oversampling achieves the largest improvement with balanced accuracy increasing by 0.14, F1-score by 0.20, and AUC by 0.17. At the variable threshold, balanced accuracy improves by 0.14 over AVGNet. Even under the moderate threshold, consistent metric gains are observed, with balanced accuracy improving by 0.01 and AUC by 0.03. As shown in Fig. 4a, e, and c, MAVG-DiffPool with BES oversampling produces superior ROC curves, with the largest gap evident under the extreme threshold.

Regularisation impact. Oversampling methods are known to be vulnerable to overfitting due to replicated minority (spikes) samples [42]. This work quantifies the risk and evaluates the effectiveness of regularisation techniques, including Dropout and weight decay (AdamW optimiser). Table 8 shows that the gap is significantly smaller when we applied regularisation techniques, indicating that regularisation effectively mitigates overfitting caused by BES oversampling. Furthermore, regularisation improves the test F1-score by 0.081 under the extreme threshold. Although regularisation improves model performance, the extreme

threshold still exhibits overfitting risk with a train-test F1-score gap of 0.594.

4.2. Comparative studies

Comparison with traditional machine learning models. Table 9 presents a comparison of the proposed framework against various baseline approaches. We first examine traditional machine learning models, including Support Vector Machine (SVM), Random Forest (RF), and Isolation Forest (iForest) variants. These models exhibit a fundamental trade-off between precision and recall at the extreme threshold. Supervised classifiers (SVM and RF) achieve high precision but low recall under extreme thresholds, missing the majority of spike events. Conversely, unsupervised iForest variants achieve high recall but extremely low precision (e.g., 0.04 at the extreme threshold), resulting in excessive false alarms. Fig. 4 further reveals these limitations: traditional supervised classifiers exhibit significantly lower ROC curves compared to the MAVG-DiffPool with BES oversampling, indicating limited discriminative ability when class imbalance is severe.

Table 8

Impact of regularisation on MAVG-DiffPool + BES using F1-score and MCC.

Spike definition	Config	F1-score			MCC		
		Train	Test	Gap	Train	Test	Gap
Extreme Threshold (IR = 107.28)	w/ reg	0.998	0.404	0.594	0.996	0.402	0.594
	w/o reg	0.986	0.323	0.663 (+0.069)	0.972	0.343	0.629 (+0.035)
Variable Threshold (IR = 52.81)	w/ reg	0.970	0.458	0.512	0.941	0.447	0.494
	w/o reg	0.991	0.418	0.573 (+0.061)	0.982	0.414	0.568 (+0.074)
Moderate Threshold (IR = 2.31)	w/ reg	0.892	0.842	0.050	0.785	0.753	0.032
	w/o reg	0.905	0.795	0.110 (+0.060)	0.807	0.693	0.114 (+0.082)

Note: Gap = Train - Test. Values in parentheses show the absolute increase in the gap without regularisation. w/ reg: with regularisation; w/o reg: without regularisation.

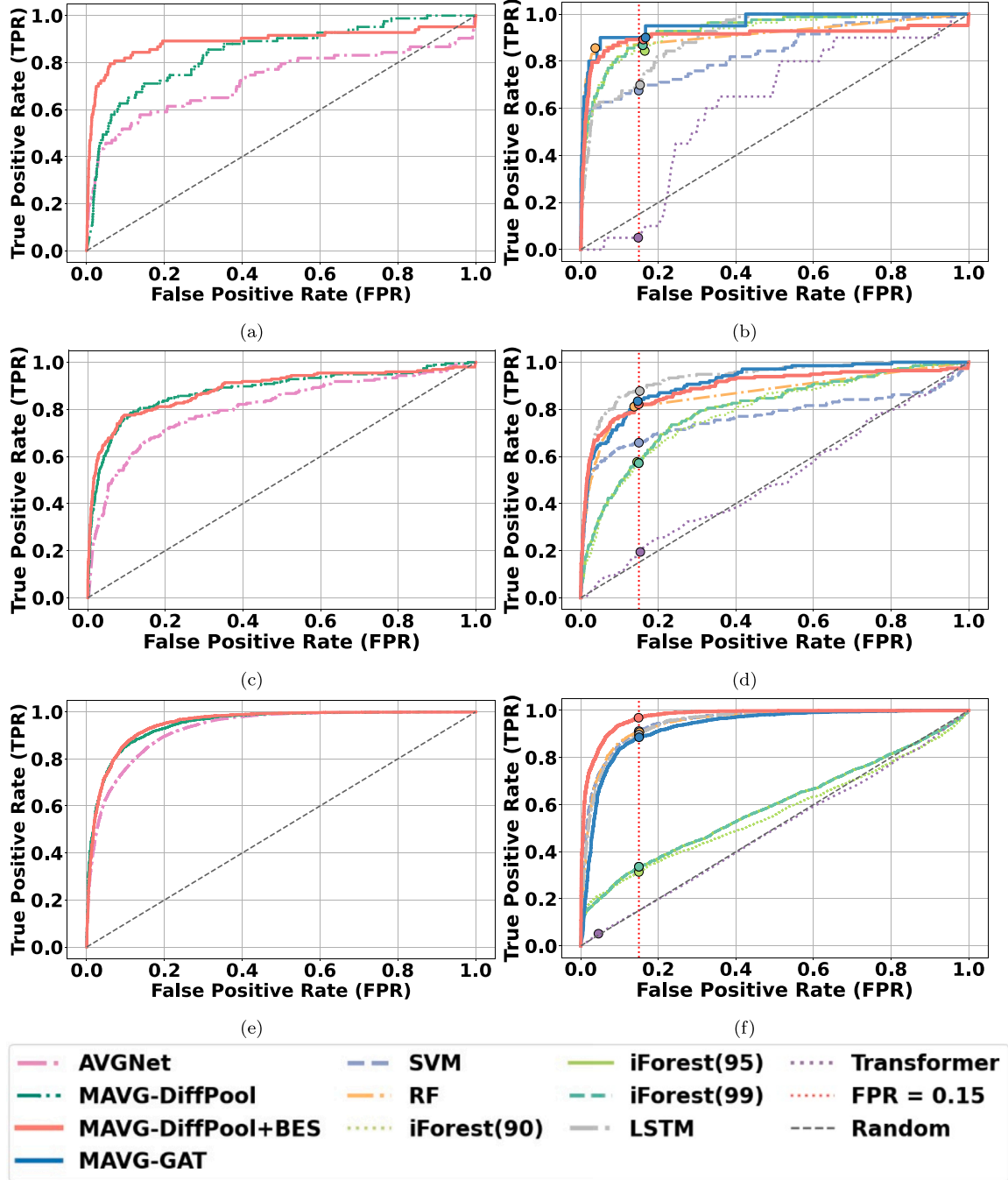


Fig. 4. Multi-panel ROC curve analysis across three spike threshold scenarios: (a, b) extreme threshold, (c, d) variable threshold, and (e, f) moderate threshold. The left panels show ablation studies of model configurations, while the right panels present a comparative analysis against baseline methods. Detailed AUC results and TPR values at $\text{FPR} \leq 0.15$ are listed in Table 9.

Table 9

Comparative results of SVM, RF, iForest variants, LSTM, Transformer, MAVG-GAT, and MAVG-DiffPool + BES for spike forecasting across extreme (IR = 107.28), variable (IR = 52.81) and moderate (IR = 2.31) thresholds.

Spike definition	Models	Bal. Acc.	Precision	Recall	F1-score	MCC	AUC	TPR (FPR ≤ 0.15)
Extreme Threshold (IR = 107.28)	SVM	0.52	0.43	0.04	0.07	0.12	0.84	0.68
	RF	0.57	0.71	0.14	0.24	0.32	0.92	0.86
	iForest(90)	0.84	0.04	0.88	0.07	0.16	0.93	0.84
	iForest(95)	0.86	0.06	0.86	0.11	0.20	0.94	0.87
	iForest(99)	0.81	0.13	0.66	0.21	0.28	0.94	0.87
	LSTM	0.56	0.83	0.12	0.21	0.32	0.91	0.70
	Transformer	0.50	0.00	0.00	0.00	0.00	0.61	0.05
	MAVG-GAT	0.63	0.46	0.25	0.32	0.34	0.96	0.90
	MAVG-DiffPool + BES	0.73	0.35	0.47	0.40	0.40	0.91	0.89
Variable Threshold (IR = 52.81)	SVM	0.56	0.47	0.13	0.21	0.24	0.77	0.66
	RF	0.59	0.50	0.19	0.27	0.30	0.87	0.81
	iForest(90)	0.72	0.07	0.64	0.12	0.16	0.79	0.58
	iForest(95)	0.71	0.09	0.56	0.15	0.18	0.80	0.57
	iForest(99)	0.64	0.14	0.32	0.20	0.19	0.80	0.57
	LSTM	0.59	0.50	0.18	0.26	0.29	0.94	0.88
	Transformer	0.50	0.00	0.00	0.00	0.00	0.50	0.19
	MAVG-GAT	0.64	0.44	0.30	0.35	0.35	0.89	0.74
	MAVG-DiffPool + BES	0.75	0.41	0.52	0.46	0.45	0.89	0.82
Moderate Threshold (IR = 2.31)	SVM	0.85	0.87	0.77	0.82	0.73	0.95	0.91
	RF	0.87	0.86	0.81	0.83	0.75	0.95	0.91
	iForest(90)	0.58	0.53	0.32	0.40	0.19	0.56	0.31
	iForest(95)	0.59	0.63	0.25	0.36	0.23	0.59	0.34
	iForest(99)	0.56	0.89	0.12	0.22	0.26	0.59	0.34
	LSTM	0.86	0.85	0.79	0.82	0.73	0.95	0.90
	Transformer	0.50	0.00	0.00	0.00	0.00	0.49	0.05
	MAVG-GAT	0.87	0.81	0.83	0.82	0.73	0.93	0.89
	MAVG-DiffPool + BES	0.88	0.79	0.89	0.84	0.75	0.97	0.97

Note: iForest(90) refers to the top rate = 0.1, iForest(95) refers to the top rate = 0.05 and iForest(99) refers to the top rate = 0.01; TPR (FPR ≤ 0.15) is the approximate True Positive Rate when False Positive Rate ≤ 0.15. IR: Imbalanced Ratio. Bal. Acc.: Balanced Accuracy. Extreme threshold corresponds to ≥ A\$300/MWh price spikes, moderate threshold to ≥ A\$100/MWh price spikes. BES: Balanced Epoch Sampler; '+' indicates the combination of methods.

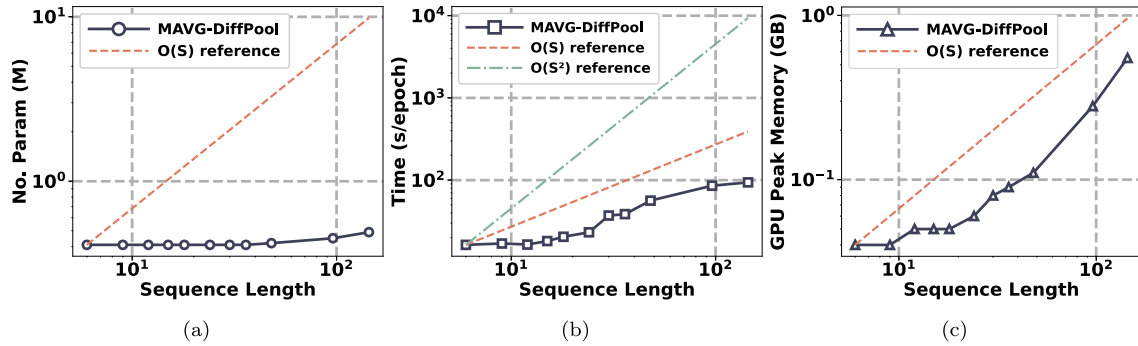


Fig. 5. Scalability analysis of MAVG-DiffPool with BES oversampling: (a) number of parameters, (b) training time, (c) GPU peak memory with sequence length S . Reference lines show $O(S)$ and $O(S^2)$ growth rates.

Comparison with sequential models. Sequential models, including Long Short-Term Memory (LSTM) networks and Transformers, leverage temporal dependencies to predict price spikes. LSTM achieves a high baseline precision (0.83) but extremely low recall (0.12). This results in poor F1-scores of only 0.21–0.26 under extreme and variable thresholds. The Transformer [32] fails completely with near-zero performance and AUC values barely above random chance (0.49–0.61), indicating that standard attention mechanisms without modelling of temporal dependencies and multivariate interdependencies are unsuitable for highly imbalanced spike forecasting tasks. Interestingly, under the variable threshold (Fig. 4d), LSTM slightly surpasses the proposed framework at higher FPR regions, due to its sequential architecture, which is naturally suited to capturing time-dependent rolling variable thresholds.

Comparison with GNN-based models. To assess the effectiveness of different graph neural network architectures, MAVG-GAT is implemented by replacing DiffPool with the Graph Attention Network (GAT) [43] layers. MAVG-GAT outperforms traditional baselines, achieving F1-score improvements of 0.11 and 0.09 over LSTM under extreme and variable thresholds. This validates that capturing multivariate interdependencies through graph structures is more effective than sequential models.

The comparison reveals an interesting pattern regarding the BES's impact across different imbalance scenarios. Under the moderate threshold, MAVG-DiffPool with BES oversampling achieves the best performance across all metrics. Under the extreme and variable thresholds with severe imbalance, MAVG-GAT achieves higher AUC values (0.96 and 0.89, respectively), while MAVG-DiffPool with BES oversampling attains superior F1-scores and MCC. This trade-off arises because aggressive oversampling improves the precision-recall balance but may lead

to overfitting to spike patterns. This reduces the ranking performance measured by AUC. DiffPool's hierarchical pooling progressively aggregates node features while preserving graph structure. Consequently, MAVG-DiffPool with BES oversampling maintains high TPR between 0.82 and 0.97 when $FPR \leq 0.15$ across all thresholds, demonstrating the effectiveness of hierarchical pooling for robust spike forecasting under varying imbalance conditions. These results demonstrate that MAVG provides an adaptive graph construction approach that maintains strong performance across different GNN architectures.

Practical implications and generalisability. Beyond demonstrating superior performance on the NEM Victoria dataset, our framework addresses two fundamental challenges in spike forecasting and anomaly detection that extend its applicability beyond this specific case study. First, the learnable visibility kernels discover spike occurrences from multivariate interdependencies without any domain-specific knowledge or manual feature engineering. Second, the balanced epoch sampler handles any severe class imbalance regardless of the domain. These design choices suggest potential applicability to other electricity markets or industrial and financial anomaly detection, though further validation in these domains is needed.

4.3. Scalability analysis

Fig. 5 presents the scalability analysis of MAVG-DiffPool with BES oversampling across sequence length S ranging from 6 to 144. Fig. 5a shows that the number of parameters exhibits nearly constant growth, remaining between 0.41 M and 0.49 M across all sequence lengths. This parameter stability, regardless of input length, demonstrates a critical advantage for deployment in resource-constrained environments. Fig. 5c shows that GPU peak memory consumption exhibits approximately linear growth, closely following the $O(S)$ reference line. Memory usage increases from 0.04GB to 0.55GB as the sequence length grows 24-fold. This near-linear scaling ensures manageable memory requirements. The model remains well below 1GB even for the longest sequences tested, which makes it suitable for deployment on standard GPU hardware. However, the training time exhibits super-linear (above $O(S)$ reference) but sub-quadratic (below $O(S^2)$ reference) growth, see in Fig. 5b. It increases from 16.31 s ($S = 12$) to 93.83 ($S = 144$) seconds per epoch. This represents the primary computational bottleneck, which may be attributed to the DiffPool soft assignment matrix computation and to the increased number of training samples from BES oversampling.

For the same sequence length $S = 12$, MAVG-DiffPool with BES oversampling has 0.41 M parameters, comparable to MAVG-GAT (0.39 M) but achieving 15 times parameter reduction compared to Transformer (6.31 M). MAVG-DiffPool with BES oversampling maintains the same memory footprint as MAVG-GAT (0.05 GB) and is 5 times more efficient than Transformer (0.26 GB). However, the training time (16.31 seconds per epoch) is slower than both MAVG-GAT (3.9 s) and Transformer (5.6 s). This reflects the combined computational overhead of DiffPool's hierarchical pooling operations and BES oversampling.

5. Conclusion

This paper proposed an innovative GNN framework for electricity price spike forecasting, where AVGNet is extended to MAVG-DiffPool - incorporating multivariate time series processing capabilities - and introduced a novel balanced epoch sampler to address the severe class imbalance inherent in spike forecasting tasks. The framework achieves an F1-score of 0.84 under the moderate threshold. Under the extreme threshold, it improves the F1-score from 0.21 (RF) to 0.40, nearly doubling the LSTM performance. A significant strength lies in its optimised precision-recall balance, directly addressing market participants' need to mitigate financial risks from missed spikes.

Nevertheless, the model has several limitations. First, precision requires further improvement to reduce false alarms in operational set-

tings. Second, the oversampling strategy introduces a risk of overfitting. Even after regularisation, the train-test F1 gap remains high (0.59) under the extreme threshold. Finally, the empirical evaluation was restricted to the NEM Victoria dataset, and applicability to other electricity markets remains to be validated.

Future works can include the following directions. First, extending the model to predict spike magnitudes. Second, mitigating overfitting through advanced oversampling strategies and/or alternative techniques such as focal loss, calibrated decision thresholds. Third, investigating the economic implications of prediction outcomes for market operation, e.g., through calibrated probabilities and cost-asymmetric thresholds. Finally, incorporating external factors such as policy changes, fuel prices, and weather variables (e.g., rainfall) to enhance robustness and applicability across diverse energy markets.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used ChatGPT in order to improve language and readability. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

CRediT authorship contribution statement

Tianshu Bao: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis; **Nariman Mahdavi:** Writing – review & editing, Supervision, Methodology; **Chris McCarthy:** Writing – review & editing, Supervision; **Dana Rezazadegan:** Writing – review & editing, Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] D. Keles, *Uncertainties in Energy Markets and Their Consideration in Energy Storage Evaluation*, 4, KIT Scientific Publishing, 2013.
- [2] J. Janczura, S. Trück, R. Weron, R.C. Wolff, Identifying spikes and seasonal components in electricity spot price data: a guide to robust modeling, *Energy Econ.* 38 (2013) 96–110.
- [3] M.Z. López, H. Zareipour, M. Quashie, Forecasting the occurrence of electricity price spikes: a statistical-economic investigation study, *Forecasting* 6 (1) (2024) 1–23. <https://doi.org/10.3390/forecast6010007>
- [4] A.E. Regulator, June 2022 Market Events Report, Market Report, Australian Energy Regulator, 2022. Accessed: 2025-05-15, <https://www.aer.gov.au/documents/aer-june-2022-market-events-report-15-december-2022>.
- [5] V.A. Kholodnyi, Derivatives use: valuation and hedging of contingent claims on power with spikes: a non-Markovian approach, *Deriv. Use, Trading Regul.* 11 (2006) 308–333. <https://doi.org/10.1057/palgrave.dutr.1840029>
- [6] A.R. Datta, S. Datta, Electricity market price-spike classification in the smart grid, in: 2016 IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT), IEEE, 2016, pp. 1–5.
- [7] D.H. Vu, K.M. Muttaqi, A.P. Agalgaonkar, A. Bouzerdoum, Short-term forecasting of electricity spot prices containing random spikes using a time-varying autoregressive model combined with kernel regression, *IEEE Trans. Ind. Inf.* 15 (9) (2019) 5378–5388.
- [8] Y.Q. Tan, Y.X. Shen, X.Y. Yu, X. Lu, Day-ahead electricity price forecasting employing a novel hybrid frame of deep learning methods: a case study in NSW, Australia, *Electr. Power Syst. Res.* 220 (2023) 109300. <https://doi.org/10.2139/ssrn.4156560>
- [9] W. Shi, Y. Wang, Y. Chen, J. Ma, An effective two-stage electricity price forecasting scheme, *Electr. Power Syst. Res.* 199 (2021) 107416. <https://doi.org/10.1016/j.epsr.2021.107416>
- [10] D. Manfre Jaimes, M. Zamudio López, H. Zareipour, M. Quashie, A hybrid model for multi-day-ahead electricity price forecasting considering price spikes, *Forecasting* 5 (3) (2023) 499–521.

- [11] N. Amjadi, F. Keynia, Electricity market price spike analysis by a hybrid data model and feature selection technique, *Electr. Power Syst. Res.* 80 (2010) 318–327.
- [12] Z. Zhou, R. Basker, D.-Y. Yeung, Graph neural networks for multivariate time-series forecasting via learning hierarchical spatiotemporal dependencies, *Eng. Appl. Artif. Intell.* 147 (2025) 110304.
- [13] D. Cheng, X. Wang, Y. Zhang, L. Zhang, Graph neural network for fraud detection via spatial-temporal attention, *IEEE Trans. Knowl. Data Eng.* 34 (8) (2020) 3800–3813.
- [14] S. Fu, X. Gao, F. Zhai, B. Li, B. Xue, J. Yu, Z. Meng, G. Zhang, A time series anomaly detection method based on series-parallel transformers with spatial and temporal association discrepancies, *Inf. Sci.* 657 (2024) 119978.
- [15] B. Jafarian, N. Yazdani, M.S. Haghighi, Using attentive temporal gnn for dynamic trust assessment in the presence of malicious entities, *Expert Syst. Appl.* 260 (2025) 125391.
- [16] A. Deng, B. Hooi, Graph neural network-based anomaly detection in multivariate time series, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 35, 2021, pp. 4027–4035.
- [17] A.-P. Surani, R. Sahetiya, Graph convolutional neural networks for optimal power flow locational marginal price, 2023, <https://arxiv.org/abs/2301.09038>.
- [18] Y. Li, C. Li, G. Chen, X. Zhou, Z. Dong, Multi-task graph adaptive learning for multivariate electricity price short-term forecasting in Australia's national electricity market, *IEEE Trans. Power Syst.* 40 (1) (2025) 530–542.
- [19] L. Lacasa, B. Luque, F. Ballesteros, J. Luque, J.C. Nuno, From time series to complex networks: the visibility graph, *Proc. Natl. Acad. Sci.* 105 (13) (2008) 4972–4975.
- [20] H. Azizi, S. Sulaimany, A review of visibility graph analysis, *IEEE Access* 12 (2024) 93517–93530.
- [21] R. Carmona-Cabezas, J. Gómez-Gómez, A.B. Ariza-Villaverde, E.G. de Ravé, F.J. Jiménez-Hornero, Multiplex visibility graphs as a complementary tool for describing the relation between ground level O₃ and NO₂, *Atmos. Pollut. Res.* 11 (1) (2020) 205–212.
- [22] V. Freitas Silva, M.E. Silva, P. Ribeiro, F. Silva, Multilayer horizontal visibility graphs for multivariate time series analysis, *Data Min. Knowl. Discov.* 39 (3) (2025) 17.
- [23] Q. Xuan, J. Zhou, K. Qiu, Z. Chen, D. Xu, S. Zheng, X. Yang, AvgNet: adaptive visibility graph neural network and its application in modulation classification, *IEEE Trans. Netw. Sci. Eng.* 9 (3) (2022) 1516–1526. <https://doi.org/10.1109/TNSE.2022.3146836>
- [24] T.M. Christensen, A.S. Hurn, K.A. Lindsay, Forecasting spikes in electricity prices, *Int. J. Forecast.* 28 (2) (2012) 400–411.
- [25] A. Rai, O. Nunn, On the impact of increasing penetration of variable renewables on electricity spot price extremes in Australia, *Econ. Anal. Policy* 67 (2020) 67–86.
- [26] X. Lu, Z.Y. Dong, X. Li, Electricity market price spike forecast with data mining techniques, *Electr. Power Syst. Res.* 73 (2005) 19–29.
- [27] T. Bao, X. Wang, N. Mahdavi, C. McCarthy, D. Rezazadegan, Dynamic XGBoost-based quantile predictor for real-time electricity price forecasting, in: *2023 IEEE International Conference on Energy Technologies for Future Grids (ETFG)*, IEEE, 2023, pp. 1–6.
- [28] S. Voronin, J. Partanen, Price forecasting in the day-ahead energy market by an iterative method with separate normal price and price spike frameworks, *Energies* 6 (11) (2013) 5897–5920.
- [29] A. Paszke, Pytorch: an imperative style, high-performance deep learning library, 2019, <https://arxiv.org/abs/1912.01703>.
- [30] E. Franczi, M. Baity-Jesi, A. Lucchi, A theoretical analysis of the learning dynamics under class imbalance, in: *International Conference on Machine Learning*, PMLR, 2023, pp. 10285–10322.
- [31] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: beyond efficient transformer for long sequence time-series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 35, 2021, pp. 11106–11115.
- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017), pp. 5998–6008.
- [33] D. Eroglu, N. Marwan, M. Stebich, J. Kurths, Multiplex recurrence networks, *Phys. Rev. E* 97 (1) (2018) 012312.
- [34] Y. LeCun, K. Kavukcuoglu, C. Farabet, Convolutional networks and applications in vision, in: *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*, IEEE, 2010, pp. 253–256.
- [35] X. Glorot, A. Bordes, Y. Bengio, Deep sparse rectifier neural networks, in: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, JMLR Workshop and Conference Proceedings, 2011, pp. 315–323.
- [36] R. Ying, J. You, C. Morris, X. Ren, W.L. Hamilton, J. Leskovec, Hierarchical graph representation learning with differentiable pooling, in: *Advances in Neural Information Processing Systems* 31 (NeurIPS 2018), 2018, pp. 4805–4815.
- [37] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, *Adv. Neural Inf. Process. Syst.* 30 (11) (2017) 1025–1035.
- [38] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: a next-generation hyperparameter optimization framework, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [39] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: *International Conference on Learning Representations*, 2019, pp. 1–8.
- [40] K.H. Brodersen, C.S. Ong, K.E. Stephan, J.M. Buhmann, The balanced accuracy and its posterior distribution, in: *2010 20th International Conference on Pattern Recognition*, IEEE, 2010, pp. 3121–3124.
- [41] S. Boughorbel, F. Jarray, M. El-Anbari, Optimal classifier for imbalanced data using matthews correlation coefficient metric, *PLoS One* 12 (6) (2017) e0177678.
- [42] K. Cao, C. Wei, A. Gaidon, N. Arechiga, T. Ma, Learning imbalanced datasets with label-distribution-aware margin loss, *Adv. Neural Inf. Process. Syst.* 32 (2019) 1565–1576.
- [43] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, et al., Graph attention networks, *Statistics* 1050 (20) (2017) 10–48550.